

LT10d : ADT Hash Table

Part 1 : About the Hash Value (or Checksum)

About Hashing ...

1. Hash function or algorithm creates a *digital fingerprint* of the data, called **digest** or simply the **hash value** (or **Checksum**).
2. Hash algorithm is primarily used for comparison purpose, not for encryption.
3. Many secure Hash Algorithms have three basic characteristics:
 - Secure : Non-reversible function
 - Fixed size : Long or short data will produce the fixed-size digest.
 - Unique* : Ideally, two different data sets cannot produce the same digest.

* not true in reality since 'collisions' do occur.

Hashing Algorithm

1. Uses the ASCII value* of the character.

Python code : `ord('a') = 97`

2. Uses a weight according to the position of the character in the data.
3. Uses the “*mod*” **modulo** operation in Computing, also known as the **modulus** in Mathematics, to find the *remainder* when a number **a** (*dividend*) is divided by another number **n** (*divisor*).

Python code : `a % n`

* <https://www.cs.cmu.edu/~pattis/15-1XX/common/handouts/ascii.html>

Example 1: A Simple Hashing Algorithm

```
def hash(string):
```

```
    total = 0
```

```
    for i in range(len(string)):
```

```
        total += ord(string[i]) * (i+1)
```

```
    return total
```

```
>>> hash('abcde') % 10
```

```
>>> 5
```

weight according to the position of the character in the string.

taking **n** to be 10

"a b c d e"

$$\begin{array}{|c|} \hline 97 \times 1 \\ \hline \end{array} + \begin{array}{|c|} \hline 98 \times 2 \\ \hline \end{array} + \begin{array}{|c|} \hline 99 \times 3 \\ \hline \end{array} + \begin{array}{|c|} \hline 100 \times 4 \\ \hline \end{array} + \begin{array}{|c|} \hline 101 \times 5 \\ \hline \end{array} = \begin{array}{|c|} \hline 1495 \\ \hline \end{array} \% \begin{array}{|c|} \hline 10 \\ \hline \end{array} = \begin{array}{|c|} \hline 5 \\ \hline \end{array}$$

When sending a message, we attach the checksum, generated from the hashing algorithm, to the end of the data.

message = **abcde5**

data checksum

When a message is received, we will generate the hash value of the data and compare it against the checksum to confirm that the data is correct.

Example 2: Another Checksum scheme

For a 5-digit data :

- a) Assign a weightage to each digit :

$$_x3 + _x5 + _x7 + _x9 + _x11 = \text{Sum}$$

- b) Apply the Sum to mod 10 :

$$\text{Sum} \% 10 = n$$

- c) Transmitted data : _ _ _ _ _ n

Example : Let the data be 12345

Compute for the Checksum :

- a) Assign a weightage to each digit :

$$1x3 + 2x5 + 3x7 + 4x9 + 5x11 = 125$$

- b) Apply the Sum to mod 10 :

$$125 \% 10 = 5$$

- c) Transmitted data : 123455

If you receive a transmitted data, 739428, can you validate if it is correct?

Checksum is used to validate the following numbers:

- NRIC number (NRIC [Validator](#) and [Checksum algorithm](#))
- Car Plate number
- ISBN number (Check digit for [ISBN-13](#) and [ISBN-10](#))
- Credit Card number ([Luhn Algorithm](#))

Checksum is used for validation of the data, **NOT** for encryption.



Singapore NRIC and Vehicle License Plate Algorithm

<https://youtu.be/XOOFoKmHR1w>